Sentiment_Analysis

A machine learning model for analyzing text for user sentiment and determine whether its a positive, neutral, or negative review.

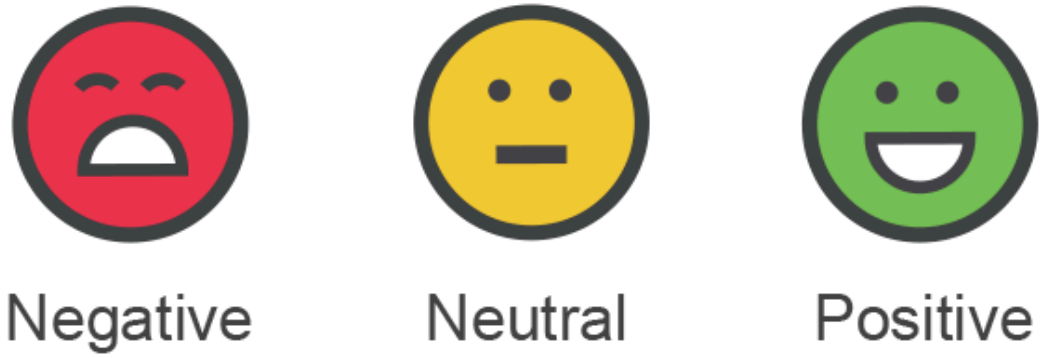
View the Project on GitHub rsanchez-dv/Sentiment_Analysis

Sentiment Analysis on Yelp's Dataset

> *Author: Roberto Sanchez, Talent Path: D1 Group*

Overview

The scope of this capstone is centered around data processing, exploratory data analysis, and training of a model to predict sentiment on user reviews.



Business Goals

Create a model to be able to be used in generating sentiment on reviews or comments found on external/internal websites to give insights on how people feel about certain topics.

This could give the company insights not easily available on sites where ratings are required or for internal use to determine sentiment on blogs or comments.

Business Applications

By utilizing this model, the business can use it for the following purposes:

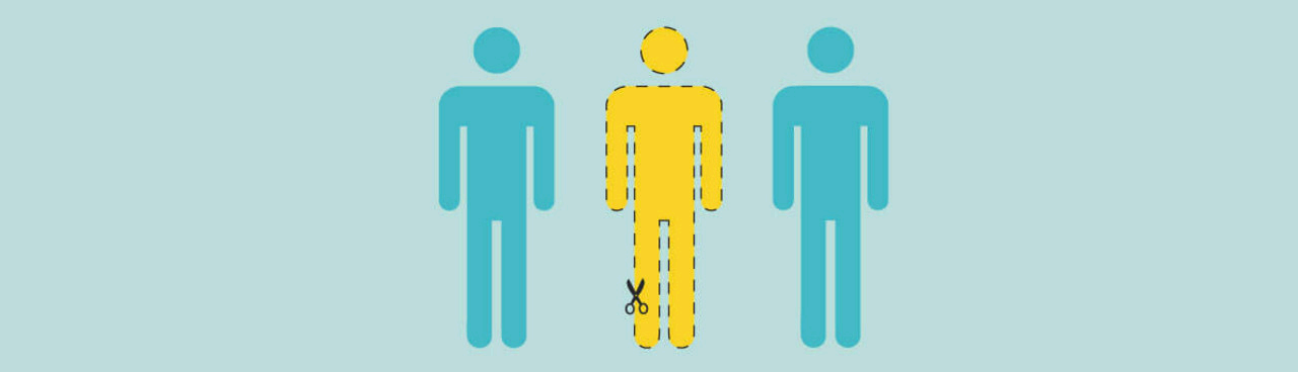
External:

- Monitoring Brand and Reputation online
- Product Research

Internal:

- Customer Support
- Customer Feedback
- Employee Satisfaction

Current methods of achieving these are using outside resources which come at a cost and increases the risk of leaking sensitive data to the public. This product will bypass these outside resources and give the company the ability to do it in-house.



Model Deployment

Link: Review Analyzer

After running multiple models and comparing accuracy, I found that the LinearSVC model is a viable candidate to be used in production for analyzing reviews of services or food.

Classification Report / Confusion Matrix:

	precision	recall	f1-score	support
0	0.84	0.88	0.86	34336
1	0.58	0.30	0.39	16318
2	0.91	0.97	0.94	99330
accuracy			0.87	149984
macro avg	0.77	0.71	0.73	149984
weighted avg	0.86	0.87	0.86	149984

	Predicted False	Predicted Neutral	Predicted True
Actual False	30114	1796	2426
Actual Neutral	4138	4881	7299
Actual True	1704	1759	95867

Technology Stack

I have been using these technologies for this project:

- Jupyter Notebook - **Version 6.3.0**
  - Used for most of the data processing, EDA, and model training.
- Python - **Version 3.8.8**
  - The main language this project will be done in.
- Scikit-learn - **Version 0.24**
  - Utilizing metrics reports and certain models.
- Postgres - **Version 13**
  - Main database application used to store this data.
- Flask - **Version 1.1.2**
  - Main backend technology to host a usable version of this project to the public.
- GitHub
  - Versioning control and online documentation

- Heroku
    - Online cloud platform to host this application for public use

Data Processing

This capstone uses the Yelp dataset found on Kaggle which comprises of multiple files:

- Business Data
- Check-in Data
- Review Data
- Tips Data
- User Data

Stage 1 - Read in From JSON files into Postgres

Overview

- Read in JSON files
- General observations on the features found in each file
- Modifying feature names to meet Postgres naming convention
- Normalized the data to prepare for import to Postgres
- Saved copies of each table as CSV file for backup incase Database goes down
- Exported data into Postgres

As stated above, Kaggle provided several JSON files with a large amount of data that needed to be stored in a location for easy access and provide a quick way to query data on the fly. As the files were read in Jupyter notebook a general observation was made to the feature names and amount each file contained to see what data I was dealing with along with the types associated with them. The business data contained a strange number of attributes that had to be broken up into separate data frames to be normalized for Postgres.

Stage 2 - Pre-Processing Data

Overview

- Read in data from Postgres
- IDing Null Values
- Removing Sparse features
- Saved data frame as a pickle to be used in model training

In this stage, I performed exploratory data analysis where I analyze any null values, see the distribution of my ratings, and review lengths.

Stage 3 - Cleaning Up Data

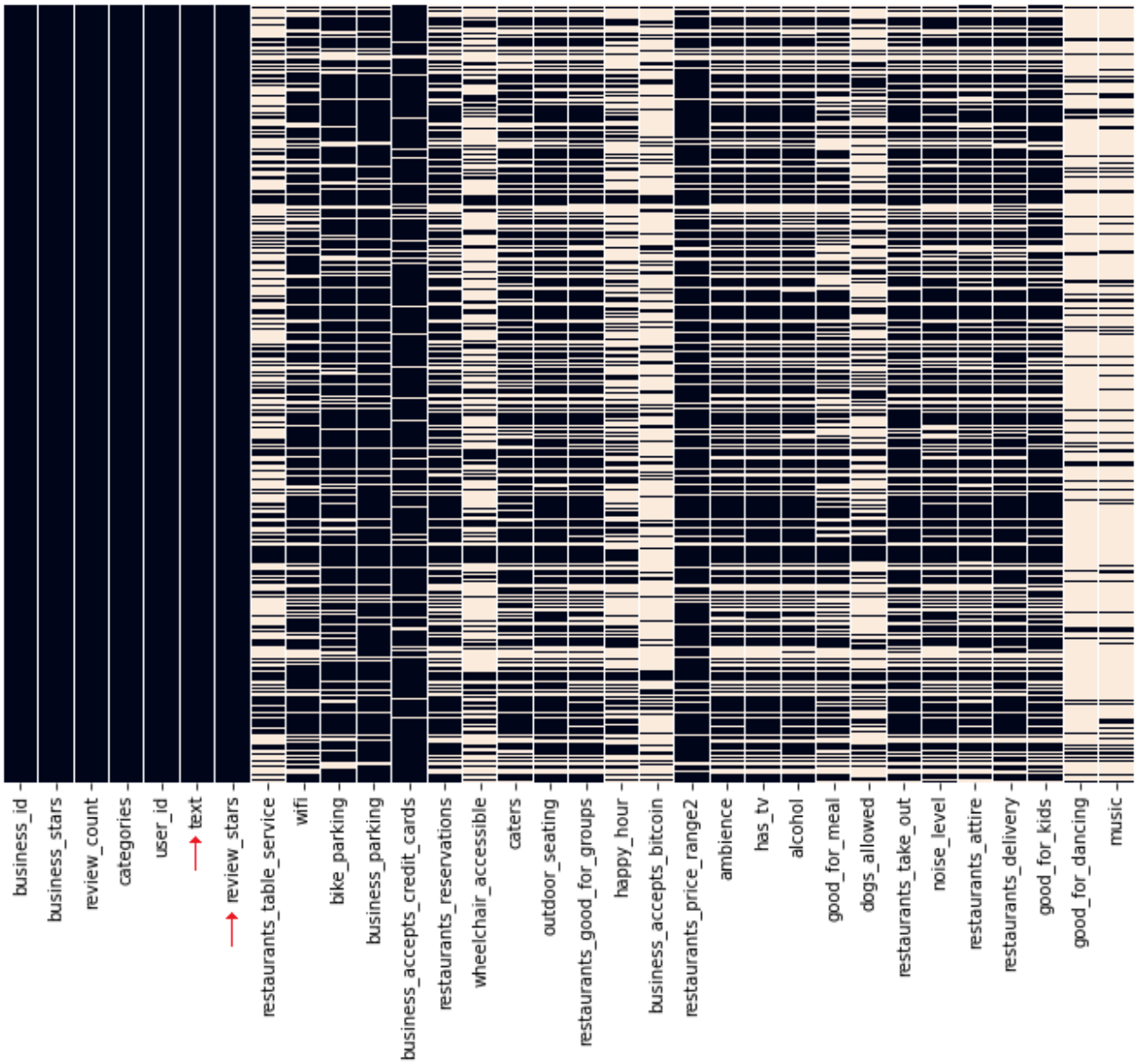
Overview

- Replace contractions with expanded versions
- Lemmatized text
- Removed special characters, dates, emails, and URLs
- Removed stop words
- Remove non-english text
- Normalized text

Exploratory Data Analysis

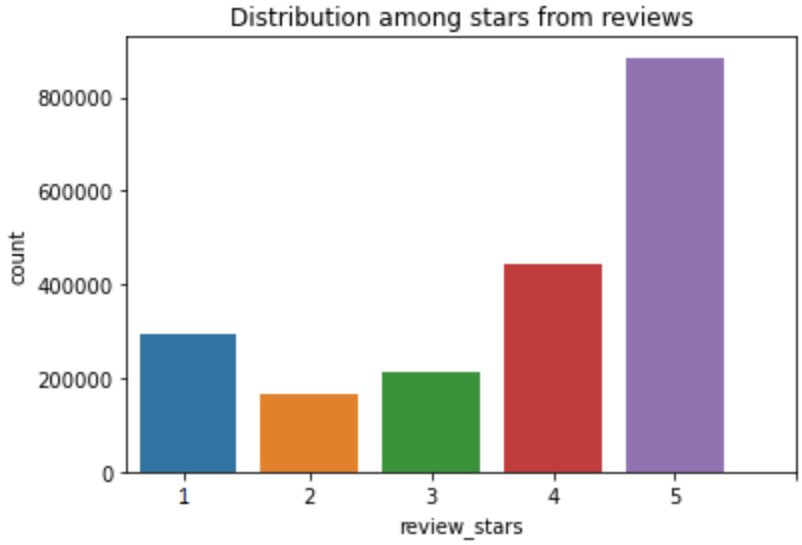
Analyzing Null Values in Dataset

Below is a visualization of the data provided by Kaggle showing which features have "NaN " values. It is clear that the review ratings (review_stars) and reviews (text) are fully populated. Some of the business attributes are sparse but have enough values to be useful for other things. Note several other features were dropped in the Data Processing since they did not provide any insights for the scope of this project.

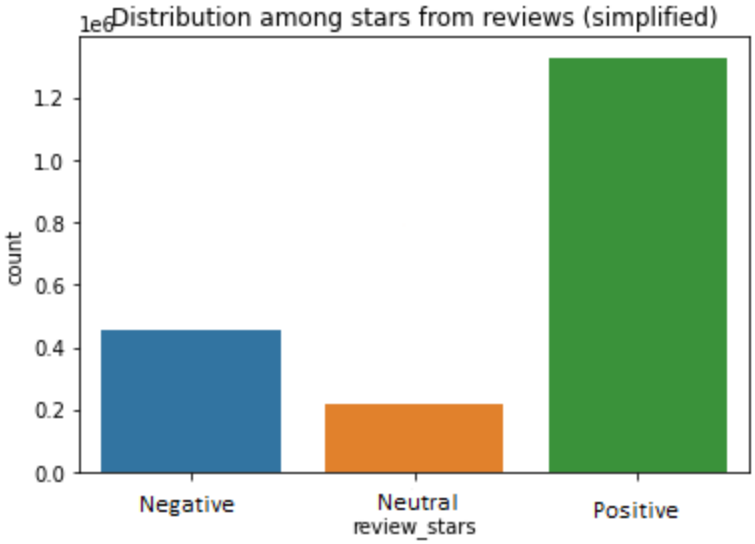


Looking Closer at the Ratings (review_stars)

This is a sample of 2 million rows from the original 8 million in the dataset. This distribution of ratings has a left skew on it where most of the reviews are 4 to 5 stars.

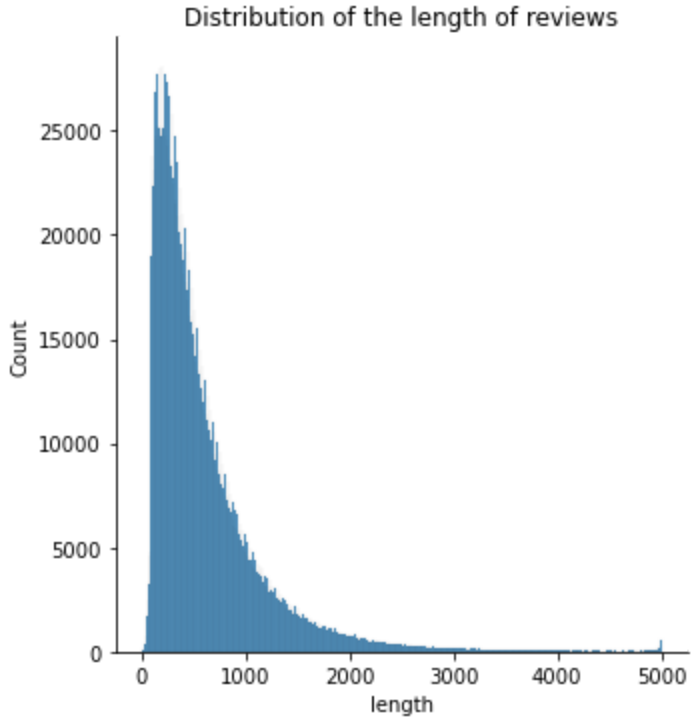


I simplified the ratings to better categorize the sentiment of the review by grouping 1 and 2 star reviews as 'negative', 3 star review as 'neutral', and 4 and 5 star reviews as 'positive'.

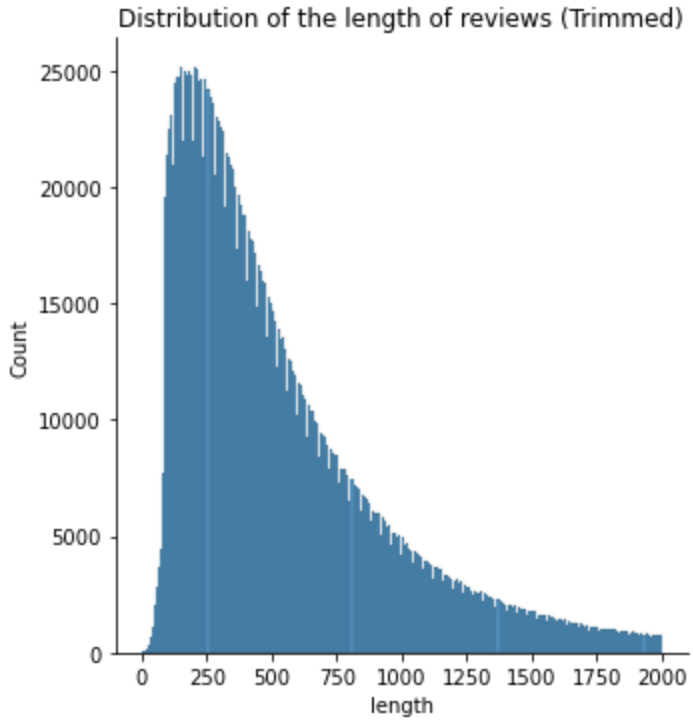


Looking Closer at the Reviews (text)

To analyze the text, I've calculated the length of each review in the sample and plotted a distribution graph showing them the number of characters of each review. The statistics were that the median review was approx. 606 characters with a range of 0 through 5000 characters.



A closer inspection on the range 0 - 2000 we can see that most of the reviews are around this general area.



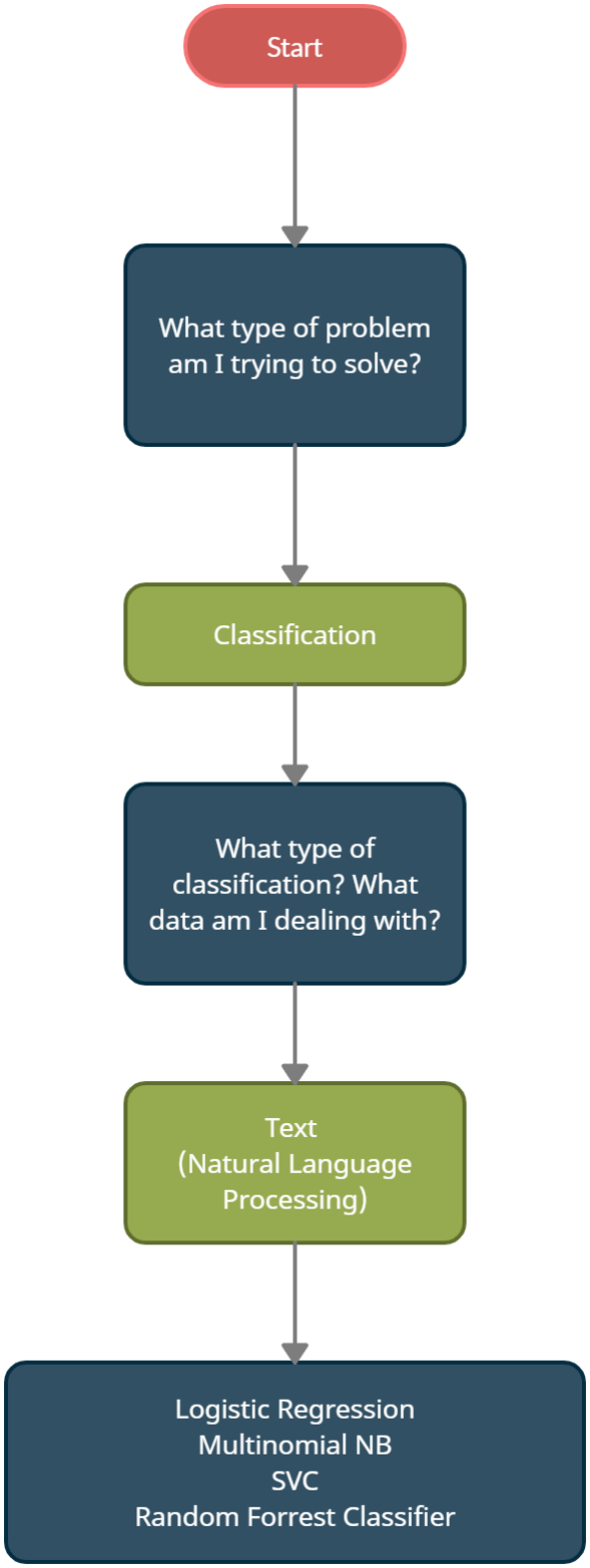
In order to produce a viable word cloud, I've had to process all of the text in the sample to remove special characters and stop words from NLTK to produce a viable string to be used in the word cloud. Below is a visualization of all of the keywords found in the positive reviews.

Word Cloud of Positive Reviews

As expected, words like "perfect", "great", "good", "great place", and "highly recommend" came out on top.

Word Cloud of Negative Reviews

customer service
told
even
go back
work
found
think
said
everything
something
looking
reason
unfortunately
issue
wish
either
went
asked
thought
mean
check
put
came back
wanted
know
probably
actually
one
fact
say
show
maybe
restaurant
tell
guy
least
used
sure
well
got
store
felt
still
understand
see
meal
left
seem
paid
even though
first time
fine
called
guess
point
given
use
kind
husband
friend
come back
front desk
last time
instead
anything
minute later
going back
location
way
yet
everyone
done
tried
look
apparently
believe
employee
course
kid
make sure
walked
coming back
right
etc
sorry
food good
told u
today
ask
find
next day
wife
owner
gave
thing
made
credit card
company
eat
cost
little
dinner
definitely
area
already
call back
case
knew
stay away
return
realized
arrived
let know
nothing
literally
establishment
needless say
showed
whatever
eating
ridiculous
ok
family
set
leave
end
lot
idea
kitchen
seemed
turned
impressed
charge
second
time
cheap
plu
hope
dog
happy
mention
expect
explained
bill
much better
served
stopped
beer
long time
life
stuff
ice cream
shop
honestly
leaving
thank
product
management
obviously
someone
menu
hotel
bought
one star
supposed
ye
needed
hand
move
cut
mind
first
informed
getting
clearly
figured
received
pizza
sort
due
usually
far
happened
offer
anyway
seriously
expensive
best
expected
year old
bar
often
spot
overall
job
feel
price
person
response
waste time
second
cheap
charge
plu
little
dinner
cost
eat
waited minute
ended
impressed
lot
end
knew
stay away
arrived
let know
return
front desk
come back
point
friend
husband
kind
use
given
every time
paid
even
told
tell
guy
least
say
actually
store
want
came back
put
go back
looking
reason
unfortunately
looked
server
issue
wish
either
went
asked
thought
mean
check
put
came back
wanted
know
probably
actually
one
fact
say
show
maybe
restaurant
tell
guy
least
used
sure
well
got
store
felt
still
understand
see
meal
left
seem
paid
even though
first time
fine
called
guess
point
given
use
kind
husband
friend
come back
front desk
last time
instead
anything
minute later
going back
location
way
yet
everyone
went



These four models were chosen to be trained with this data. Each of these models had a pipeline created with TfidfVectorizer.

Model Training

- Run a StratifiedKFold with a 5 fold split and analyze the average scores and classification reports
    - Get an average accuracy of the model for comparison

- Create a single model to generate a confusion matrix
- Test out model on a handful of examples

Below is the average metrics after running 5 fold cross validation on LinearSVC

```
accuracy: 87.52% (+/- 0.03%)
precision: 77.61% (+/- 0.14%)
recall: 71.73% (+/- 0.11%)
f1 score: 73.22% (+/- 0.13%)
Wall time: 25min 55s
```

Testing Model

After the model was trained, I fed it some reviews I found online to test out whether or not the model can properly detect the right sentiment. The following reviews are ordered as "Negative", "Neutral", and "Positive":

```
new_test_data = [
    "This was the worst place I've ever eaten at. The staff was ru
    "It was ok, I guess",
    "I had a pleasent time with kimberly at the granny shack. The
]
res = model.prediction(new_test_data)
```

Below is a screenshot of the results when feed into the LinearSVC model for predictions.

```
🧠 Predicts: 😡 Negative Review
        💬 "This was the worst place I've ever eaten at. The staff was rude and did not take my order
until after i pulled out my wallet."

🧠 Predicts: 😐 Neutral Review
        💬 "It was ok, I guess"

🧠 Predicts: 🙂 Positive Review
        💬 "I had a pleasent time with kimberly at the granny shack. The food was amazing and very fa
mily friendly."
```

End Notes

There are some improvements to be made such as the following:

- **Balancing the data**
  - This can be seen in the confusion matrix for the candidate models and other models created that the predictions come out more positive than negative or neutral.
  - While having poor scores in the neutral category, the most important features are found in the negative and positive predictions for business applications.

- **Hyper-parametrization improvement**
  - Logistic Regression and Multinomial NB models produced models within a reasonable time frame while returning reasonable scores. Random Forrest Classifier and SVM took a **significant** amount of time to produce just one iteration. In order to produce results from this model, StratifiedKFold was not used in these two models. Changing SVM to LinearSVC improved performance dramatically and replaced the SVM model and outperformed the Logistic Regression model by a small margin which was the original candidate model.

This project is maintained by rsanchez-dv

Hosted on GitHub Pages — Theme by orderedlist